Un modèle qui dessine directement en espace écran

Proposition d'orientation et plan d'action

Melissa Colin — projet « Graph drawing différentiable », ENSEIRB-MATMECA 3A

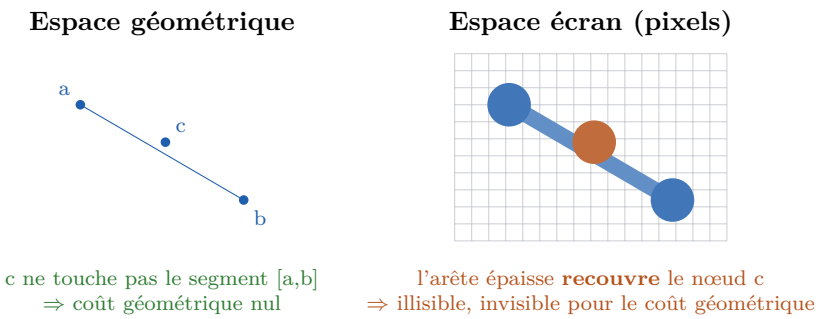
9 octobre 2026 — *document de travail, à discuter avec l'équipe et M. Giot*

En bref

- **Idée** : entraîner un réseau de neurones sur graphes qui, à partir d'un graphe, **prédit directement la position de chaque nœud dans l'image finale** (coordonnées écran), au lieu d'optimiser chaque graphe séparément.
- **Entraînement** : avec des fonctions de coût calculées **sur l'image rendue** (rendu différentiable par SDF), donc sur ce que l'œil voit réellement.
- **Bonus de fin de projet** : affiner le modèle selon des **préférences visuelles humaines** (critère non codable), via un modèle critique appris.
- **Ce que ça change dans l'organisation** : on travaille **en parallèle** sur le modèle, sur le rendu et les coûts, et sur l'évaluation, avec une interface commune fixée dès le départ.

1 Le problème, en une figure

Les méthodes classiques ((GD)^2, SGD, FORBID) optimisent un dessin dans un **espace géométrique** continu : un nœud est un point, une arête est un segment d'épaisseur nulle. Mais le dessin est finalement **affiché en pixels**, avec des nœuds de taille fixe, des arêtes épaisses et une résolution limitée. Un dessin « bon » en géométrie peut donc être illisible à l'écran.

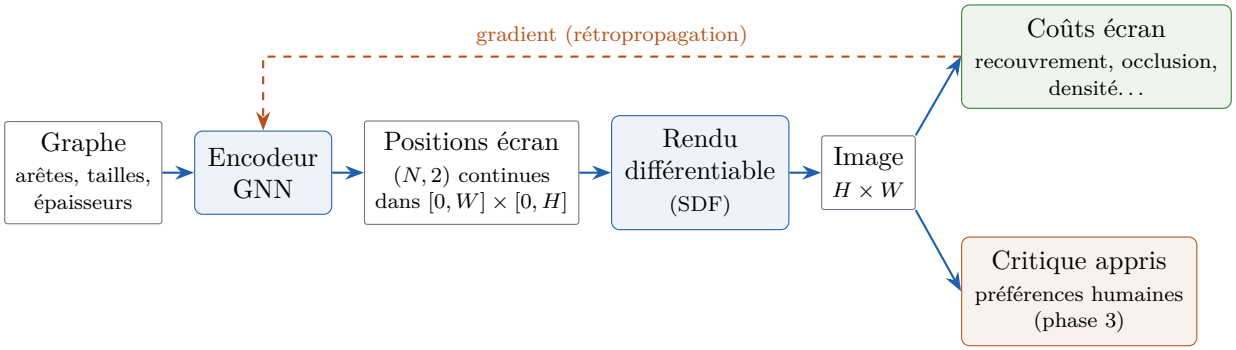


Le sujet propose de calculer les coûts **sur l'image rendue**. Pour que le gradient remonte des pixels jusqu'aux positions, le rendu doit être **différentiable** : chaque nœud et chaque arête est décrit par une fonction de distance signée (SDF) $d(p)$, transformée en intensité douce $I(p) = \sigma(-d(p)/\varepsilon)$.

2 L'idée proposée

2.1 Un modèle qui pense en pixels

Plutôt que d'optimiser les positions de *chaque* graphe par descente de gradient (coûteux à chaque nouveau graphe), on entraîne **une fois** un réseau qui produit le dessin **directement** :



— **Entrée** : la structure du graphe et ses attributs (taille des nœuds, épaisseur des arêtes). Un GNN gère naturellement un nombre de nœuds variable.

— **Sortie** : des coordonnées **continues** en pixels. On commence avec une taille d'image fixe, puis on étudie comment passer la résolution en entrée du modèle.

— **Contraintes visuelles intégrées** : idéalement, l'architecture garantit certaines contraintes (par exemple l'absence de chevauchement) plutôt que de seulement les pénaliser.

2.2 Trois phases d'entraînement



Phase 3 en détail. On annote des **paires** de dessins (« je préfère A à B »), on entraîne un **modèle critique** (par exemple un CNN sur l'image) à prédire ces préférences, puis on affine le modèle de dessin pour maximiser ce score, en le gardant proche du modèle de la phase 2 (pénalité de divergence) pour qu'il ne « triche » pas. Trois façons de faire, à comparer :

— **RL** (PPO / REINFORCE), comme le RLHF des modèles de langage : le modèle sort une distribution sur les positions, on tire des dessins, le critique les note ;

— **DPO** : apprendre directement à partir des paires de préférences, sans critique ni boucle de RL ; plus stable avec peu de données ;

— **rétropropagation directe** : si le critique et le rendu sont différentiables, on peut rétropropager son score jusqu'au modèle (approche de type DRaFT, nettement plus efficace que le RL quand c'est possible).

3 Lien avec les axes du sujet

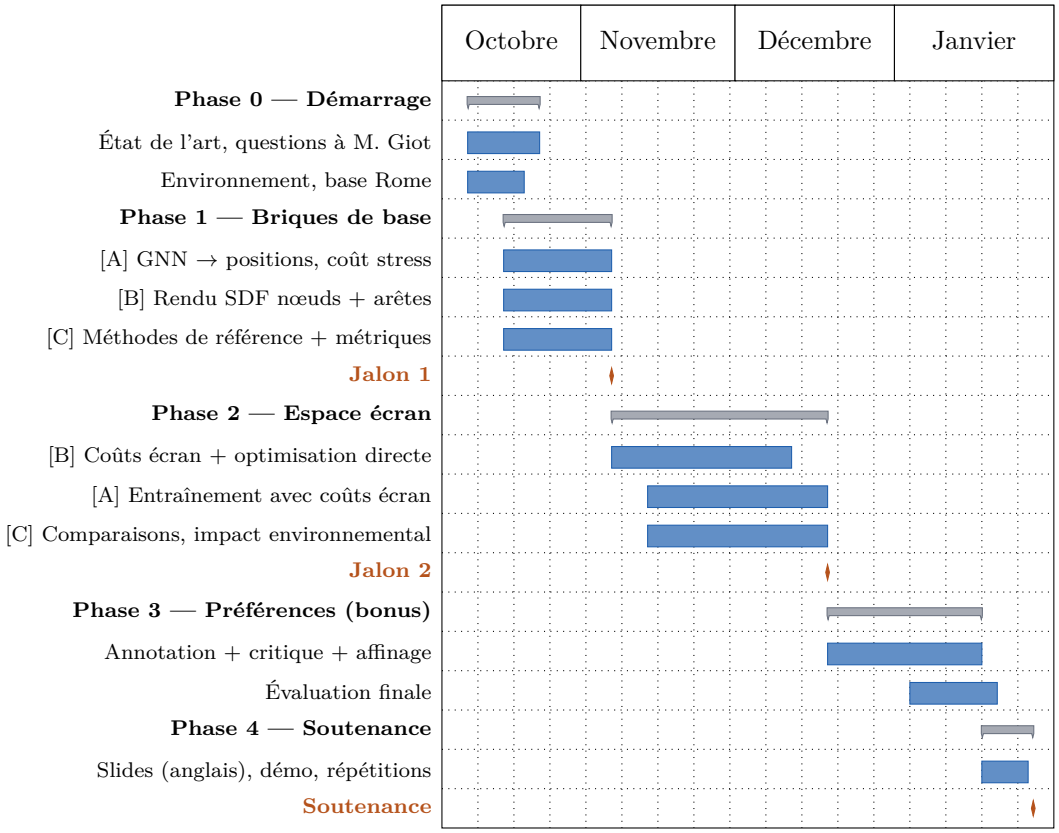
Axe du sujet	**Rôle dans cette proposition**
Rendu différentiable + coûts écran	**Indispensable** : c'est ce qui entraîne le modèle (phase 2) et ce que regarde le critique (phase 3).
Optimisation directe par gradient	Sert de **référence** : pour un graphe donné, la meilleure solution qu'on sait obtenir avec ces coûts. Le modèle doit s'en approcher.
Modèle profond (axe « ambitieux »)	**Cœur de la proposition**, avec la sortie en coordonnées écran et l'affinage par préférences.
Comparaison aux méthodes classiques	Inchangée : on mesure le gain ou la perte par rapport aux dessins géométriques.

4 Points de vigilance et parades

Risque	**Parade envisagée**
Pixels distincts $\neq$ pas de chevauchement : un nœud est un disque de r pixels ; deux nœuds sur des pixels voisins se recouvrent quand même.	La contrainte réelle est $\|c_i - c_j\| \geq r_i + r_j$. À traiter par un coût de recouvrement sur l'image, ou par une couche de projection / répulsion en sortie du réseau.
Une sortie discrète bloque le gradient : choisir un pixel revient à un arg max, de gradient nul.	Sortir des coordonnées **continues** (ou des cartes de probabilité douces, Gumbel-softmax). Voir une affectation « un nœud par case » comme un problème d'affectation différentiable (Sinkhorn).
Les arêtes ne sont pas prédites : ce sont des segments droits entre nœuds ; occlusions et fusions de traits viennent d'elles.	Le modèle les contrôle **indirectement** via les nœuds ; les coûts sur l'image (occlusion arête-nœud, densité) font remonter le gradient vers les extrémités.
Mémoire et calcul : de l'ordre de $(N+E) \times H \times W$ par graphe.	Petits graphes (base Rome), résolution modeste au début ; calcul des recouvrements via la carte somme $S = \sum_i I_i$ plutôt que paire par paire.
Préférences humaines : il faut beaucoup d'annotations, et trois annotateurs ont des goûts biaisés.	Phase 3 traitée comme un **bonus** ; DPO ou critique préentraîné sur des métriques connues (lisibilité, cf. Haleem et al.) pour réduire le besoin d'annotations.
Évaluer un modèle appris est plus long qu'une optimisation directe.	Une personne dédiée à l'évaluation et aux méthodes de référence dès le début.

5 Plan d'action

5.1 Calendrier



5.2 Livrables visés par jalon

06/11 État de l'art synthétisé ; cette proposition validée (ou réorientée) avec M. Giot ; première image d'un graphe rendue par SDF et différentiable ; premier GNN qui sort des positions.

18/12 Au moins deux coûts écran ; optimisation directe d'un graphe en espace écran ; modèle entraîné avec ces coûts ; première comparaison chiffrée aux méthodes classiques.

27/01 Résultats et comparaison finale ; phase 3 si le temps le permet ; analyse d'impact environnemental ; démo.

5.3 Répartition proposée (à discuter)

Rôle	Responsabilité
A — Modèle	Architecture GNN (graphe → positions écran), entraînement supervisé puis avec coûts écran, phase 3.
B — Rendu et coûts	Rendu différentiable (SDF ou autre méthode si plus adaptée), coûts écran, optimisation directe d'un graphe.
C — Données et évaluation	Base Rome, méthodes de référence classiques, métriques, comparaisons, mesure d'impact environnemental.

Interface commune, à figer dès la phase 0, pour que chacun avance sans attendre les autres :

— le modèle produit `positions` : tenseur $(N, 2)$, valeurs continues dans $[0, W] \times [0, H]$;

— `render(positions, graphe, attributs)` → `image` (C, H, W), différentiable ;

— chaque coût : `loss(image, ...)` → `scalaire`.

En attendant le rendu (B), le rôle A s'entraîne avec un coût géométrique simple (stress) pour valider son architecture.

6 Questions pour M. Giot

1. Cette orientation (modèle appris en coordonnées écran, coûts écran comme signal d'entraînement) vous paraît-elle compatible avec vos attentes, ou préférez-vous qu'on commence par l'optimisation directe ?
2. Qu'attendez-vous concrètement aux jalons du 06/11 et du 18/12 ?
3. Les « exemples de code » mentionnés dans le sujet : comment y accéder ?
4. Ressources de calcul disponibles (GPU au LaBRI ou à l'école) ?
5. GitHub ou GitLab pour le dépôt ?

7 Ressources pour creuser

Modèles appris pour le dessin de graphes

— X. Wang, K. Yen, Y. Hu, H.-W. Shen. *DeepGD : A Deep Learning Framework for Graph Drawing Using GNN.* IEEE CG&A, 2021. — GNN entraîné directement sur des critères esthétiques.

— X. Wang et al. *SmartGD : A GAN-Based Graph Drawing Framework for Diverse Aesthetic Goals.* IEEE TVCG, 2023. — un discriminateur apprend ce qu'est un « bon » dessin.

— L. Giovannangeli et al. *Toward Efficient Deep Learning for Graph Drawing (DL4GD).* IEEE TVCG. — référence de l'encadrant.

Juger un dessin à partir de son image (base du critique)

— H. Haleem, Y. Wang, A. Puri, S. Wadhwa, H. Qu. *Evaluating the Readability of Force Directed Graph Layouts : A Deep Learning Approach.* IEEE CG&A 39(4), 2019. — un CNN prédit des métriques de lisibilité à partir de l'image du graphe.

Rendu différentiable

— I. Quilez. *2D distance and gradient functions*, 2019. `https://iquilezles.org/articles/distgradfunctions2d/`

— T.-M. Li, M. Lukáč, M. Gharbi, J. Ragan-Kelley. *Differentiable Vector Graphics Rasterization for Editing and Learning (DiffVG).* ACM TOG, 2020. — rasteriseur différentiable de formes vectorielles, alternative possible aux SDF.

Optimisation géométrique (méthodes de référence)

— J. X. Zheng, S. Pawar, D. F. M. Goodman. *Graph Drawing by Stochastic Gradient Descent.* IEEE TVCG, 2018.

— R. Ahmed et al. *Graph Drawing via Gradient Descent, $(GD)^2$.* Graph Drawing, 2020.

— L. Giovannangeli et al. *FORBID : Fast Overlap Removal By Stochastic Gradient Descent for*

Graph Drawing. Graph Drawing, 2022.

Affinage par préférences humaines

— P. Christiano et al. *Deep Reinforcement Learning from Human Preferences.* NeurIPS, 2017. — l'idée fondatrice : apprendre une récompense à partir de comparaisons par paires.

— L. Ouyang et al. *Training Language Models to Follow Instructions with Human Feedback (InstructGPT).* NeurIPS, 2022. — la recette supervisé → critique → RL.

— R. Rafailov et al. *Direct Preference Optimization.* NeurIPS, 2023. — sans critique ni RL.

— K. Clark, P. Vicol, K. Swersky, D. J. Fleet. *Directly Fine-Tuning Diffusion Models on Differentiable Rewards (DRaFT).* ICLR, 2024. — rétropropager une récompense différentiable plutôt que faire du RL.

Sorties discrètes différentiables

— E. Jang, S. Gu, B. Poole. *Categorical Reparameterization with Gumbel-Softmax.* ICLR, 2017.

— G. Mena, D. Belanger, S. Linderman, J. Snoek. *Learning Latent Permutations with Gumbel-Sinkhorn Networks.* ICLR, 2018.